

Notes: Indexes (2)

All of the problems assume the following schema.

```
CREATE TABLE users (  
    id_users BIGINT PRIMARY KEY,  
    created_at TIMESTAMPTZ,  
    username TEXT UNIQUE NOT NULL  
);  
  
CREATE TABLE tweets (  
    id_tweets BIGINT UNIQUE NOT NULL,  
    id_users BIGINT REFERENCES users(id_users),  
    in_reply_to_user_id BIGINT REFERENCES users(id_users),  
    created_at TIMESTAMPTZ,  
    text TEXT  
);  
  
CREATE TABLE tweet_tags (  
    id_tweets BIGINT REFERENCES tweets(id_tweets),  
    tag TEXT,  
    PRIMARY KEY(id_tweets, tag)  
);
```

1. Consider the following SQL query.

```
SELECT id_tweets
FROM tweets
WHERE to_tsvector(text) @@ to_tsquery(:tsquery)
      AND created_at > '2020-01-01'
ORDER BY created_at <=> '2020-01-01'
LIMIT 10;
```

- a) Create index(es) so that the following query can use an index scan, avoid an explicit sort, and take advantage of the LIMIT clause for faster processing.

Do not create any unneeded indexes; if no new indexes are needed, say so.

- b) Assume that you are unable to install the RUM extension into the database. (This could be because you don't have admin permissions, for example.) How would that change your answer above?

2. Consider the following SQL query.

```
SELECT *
FROM (
    SELECT
        id_tweets,
        unnest(tsvector_to_array(to_tsvector(text))) as lexeme
    FROM tweets
) t
WHERE lexeme = :lexeme;
```

- a) The query above cannot be sped up using an index. Why?
- b) Rewrite the query from the previous question into an equivalent query that can be sped up using an index. Also provide the index that would speed up the query.

3. Consider the following SQL query that uses a cross join.

```
SELECT id_tweets
FROM users, tweets
WHERE to_tsvector(text) @@ to_tsquery(username)
      AND users.id_users = :id_users;
```

- a) Which join methods can be used to implement this query?
- b) If it is possible to construct an index that will speed up this query, do so. Otherwise, state that it is impossible and explain why.